

National Institute of Technology Karnataka Surathkal
Department of Information Technology



IT 252 DATABASE SYSTEMS

The Relational Algebra and Relational Calculus

Dr. Jayashree T R

Course Outline

Course Plan: Theory:

Part A: Parallel Computer Architectures

Week 1: Introduction to the course, highlighting the data and databases, and related basic concepts. Advantages and need for a database system.

Week 2: Attributes, tuples, relational schema, conceptual model, introduction to SQL-DML, DDL, creating relations/ tables, access and manipulate data.

Week 3,4: Data model, relational model, and SQL, relational data model and relational database constraints, SQL data definition and data types, specifying constraints, basic retrieval queries, complex queries, triggers, views, schema modification.

Week 5-6: ER model development, entity types/sets, attribute, relationship types/sets, simple employee database conceptual design using ER concepts, ER to relational mapping algorithm, EER model.

Course Outline

- Week 7,8 : Functional dependency definition, need of normalization, anomalies and redundant information in tuples, normalization steps (1NF, 2NF, 3NF), BCNF, introduction to the higher normal forms.
- Week 9: Procedural query languages: relational algebra and SQL-SELECT, PROJECT, RENAME, binary and JOIN operations with examples.
Non-Procedural query languages: relational calculus, and SQL-tuple relational calculus, existential and universal quantifier with examples.
- Week 10,11 : Transaction management, schedule and serializability, concurrency control, 2-phase lock, recovery mechanism: undo/redo values.
- Week 12,13: Disk storage- basic file structures, ordered/unordered. binary search, hashing, indexing, importance of indexes, primary and secondary indexing methods, clustered, B tree, B++ trees.
- Week 14: Current trends in database system, introduction to data warehousing, data mining.
- Practical: MySQL commands
- Project: Team of 2 or 3 members

Chapter Outline

- Relational Algebra
 - Unary Relational Operations
 - Relational Algebra Operations From Set Theory
 - Binary Relational Operations
 - Additional Relational Operations
 - Examples of Queries in Relational Algebra
- Relational Calculus
 - Tuple Relational Calculus
 - Domain Relational Calculus
- Example Database Application (COMPANY)

The Relational Algebra and Relational Calculus

- The RELATIONAL ALGEBRA and the RELATIONAL CALCULUS  FORMAL LANGUAGES
- SQL standard  PRACTICAL LANGUAGE
- The relational algebra and calculus were developed **before the SQL** language. SQL is primarily based on concepts from relational calculus and has been extended to incorporate some concepts from relational algebra as well.
- RELATIONAL ALGEBRA is the basis for **SEQUEL** (Structured English Query Language), later shortened to **SQL**.

- **Relational algebra**
 - Basic set of operations for the relational model
- **Relational algebra expression**
 - Sequence of relational algebra operations
- **Relational calculus**
 - Higher-level declarative language for specifying relational queries

Relational Algebra Overview

- Relational algebra is the basic set of operations for the relational model
- These operations enable a user to specify **basic retrieval requests** (or **queries**)
- The result of an operation is a *new relation*, which may have been formed from one or more *input relations*
 - This property makes the algebra “closed” (all objects in relational algebra are relations)

Relational Algebra Overview (continued)

- The **algebra operations** thus produce new relations
 - These can be further manipulated using operations of the same algebra
- A sequence of relational algebra operations forms a **relational algebra expression**
 - The result of a relational algebra expression is also a relation that represents the result of a database query (or retrieval request)

Brief History of Origins of Algebra

- Muhammad ibn Musa al-Khwarizmi (800-847 CE) – from Morocco wrote a book titled al-jabr about arithmetic of variables
 - Book was translated into Latin.
 - Its title (al-jabr) gave Algebra its name.
- Al-Khwarizmi called variables “shay”
 - “Shay” is Arabic for “thing”.
 - Spanish transliterated “shay” as “xay” (“x” was “sh” in Spain).
 - In time this word was abbreviated as x.
- Where does the word Algorithm come from?
 - Algorithm originates from “al-Khwarizmi”
 - Reference: PBS (<http://www.pbs.org/empires/islam/innoalgebra.html>)

Relational Algebra Overview

- Relational Algebra consists of several groups of operations
 - Unary Relational Operations
 - SELECT (symbol: σ (sigma))
 - PROJECT (symbol: π (pi))
 - RENAME (symbol: ρ (rho))
 - Relational Algebra Operations From Set Theory
 - UNION ($\cup$), INTERSECTION ($\cap$), DIFFERENCE (or MINUS, $-$)
 - CARTESIAN PRODUCT ($\times$)
 - Binary Relational Operations
 - JOIN (several variations of JOIN exist)
 - DIVISION
 - Additional Relational Operations
 - OUTER JOINS, OUTER UNION
 - AGGREGATE FUNCTIONS (These compute summary of information: for example, SUM, COUNT, AVG, MIN, MAX)

Mathematical Foundations: Cartesian Product

- Let:
 - A be the set of values $\{a_1, a_2, \dots\}$
 - B be the set of values $\{b_1, b_2, \dots\}$
 - C be the set of values $\{c_1, c_2, \dots\}$
- The Cartesian product of A and B (written $A \times B$) is the set of all possible ordered pairs (a_i, b_j) , where $a_i \in A$ and $b_j \in B$.
- Similarly:
 - $A \times B \times C$ is the set of all possible ordered triples (a_i, b_j, c_k) , where $a_i \in A$, $b_j \in B$, and $c_k \in C$.
 - $A_1 \times A_2 \times \dots \times A_n$ is the set of all possible ordered *tuples* $(a_{1i}, a_{2j}, \dots, a_{nk})$, where $a_{de} \in A_d$

Cartesian Product Example

- $A = \{\text{small, medium, large}\}$
- $B = \{\text{shirt, pants}\}$

A X B	Shirt	Pants
Small	(Small, Shirt)	(Small, Pants)
Medium	(Medium, Shirt)	(Medium, Pants)
Large	(Large, Shirt)	(Large, Pants)

- $A \times B = \{(\text{small, shirt}), (\text{small, pants}), (\text{medium, shirt}), (\text{medium, pants}), (\text{large, shirt}), (\text{large, pants})\}$

Example: Cartesian Product

- What is the Cartesian Product of $A \times B$?
 - $A = \{\text{perl, ruby, java}\}$
 - $B = \{\text{necklace, ring, bracelet}\}$
- What is $B \times A$?

A x B	Necklace	Ring	Bracelet
Perl	(Perl,Necklace)	(Perl, Ring)	(Perl,Bracelet)
Ruby	(Ruby, Necklace)	(Ruby,Ring)	(Ruby,Bracelet)
Java	(Java, Necklace)	(Java, Ring)	(Java, Bracelet)

Mathematical Foundations: Relations

- The domain of a variable is the set of its possible values
- A relation on a set of variables is a subset of the Cartesian product of the domains of the variables.
 - Example: let x and y be variables that both have the set of non-negative integers as their domain
 - $\{(2,5),(3,10),(13,2),(6,10)\}$ is one relation on (x, y)
- A table is a subset of the Cartesian product of the domains of the attributes. Thus a **table is a mathematical relation**.
- Synonyms:
 - Table = relation
 - Row (record) = tuple
 - Column (field) = attribute

Mathematical Relations

- In tables, as, in mathematical relations, the order of the tuples does not matter but the order of the attributes does.
- The domain of an attribute usually includes NULL, which indicates the value of the attribute is unknown.

What is an Algebra?

- Mathematical system consisting of:
- Operands --- variables or values from which new values can be constructed.
- Operators --- symbols denoting procedures that construct new values from given values.

What is Relational Algebra?

- An algebra whose operands are relations or variables that represent relations.
- Operators are designed to do the most common things that we need to do with relations in a database.
- The result is an algebra that can be used as a query language for relations.

Relational Algebra

- A collection of operations that users can perform on relations to obtain a desired result
- This is an introduction and only covers the algebra needed to represent SQL queries
 - Select, project, rename
 - Cartesian product
 - Joins (natural, condition, outer)
 - Set operations (union, intersection, difference)
- Relational Algebra treats relations as sets: duplicates are removed

Relational Algebra

One or
more
relations



Operation



Resulting
Relation

For each operation:
both the operands and the result are
relations

Basic Relational Algebra Operations

- Basic operations:
 - **Selection** (σ): Selects a subset of tuples from a relation.
 - **Projection** (π): Selects columns from a relation.
 - **Cross-product** ($\times$): Allows us to combine two relations.
 - **Set-difference** ($-$): Tuples in relation 1, but not in relation 2.
 - **Union** ($\cup$): Tuples in relation 1 and in relation 2.
- Additional operations:
 - Intersection, join, division, renaming: Not essential, but (very) useful.
- Each operation returns a relation, operations can be **composed** (Algebra is “closed”)
 - Contains the closure property
- Since operators' input is a relation and its output is a relation we can string these operators together to form a more complex operator

Basic Operation: Projection

 π

- Deletes attributes that are not in projection list.
- Schema of result contains exactly the fields in the projection list, with the same names that they had in the input relation.
- Syntax: $\pi_{f_1, f_2, \dots}$ (Relation)
- Projection operator has to eliminate duplicates.
(Why?)
 - Note: real systems typically do not eliminate duplicates unless the user explicitly asks for it.
(Why not?)

SID	Name	Login	DoB	GPA
55515	Smith	smith@ccs	Jan 10, 1990	3.82
55516	Jones	jones@hist	Feb 11, 1992	2.98
55517	Ali	ali@math	Sep 22, 1989	3.11
55518	Smith	smith@math	Nov 30, 1991	3.32

 $\pi_{Sid, Name}(S1)$

SID	Name
55515	Smith
55516	Jones
55517	Ali
55518	Smith

 $\pi_{Sid}(S1)$

SID
55515
55516
55517
55518

Consider the relation STUDENT:

Now, if you apply the **projection** operator to select the **Name** column from the relation, $(\pi_{\text{Name}}(\text{STUDENT}))$:

STUDENT

StudentID	Name	Age
1	Alice	22
2	Bob	23
3	Alice	22

Name
Alice
Bob

Appears only once in the result of projection operation.

Projection eliminates duplicates because a relation in relational algebra is considered a **set of tuples**, and **sets *do not* allow duplicate elements**.

Can you compare the **Projection** operation with the **SELECT** statement of MySQL?

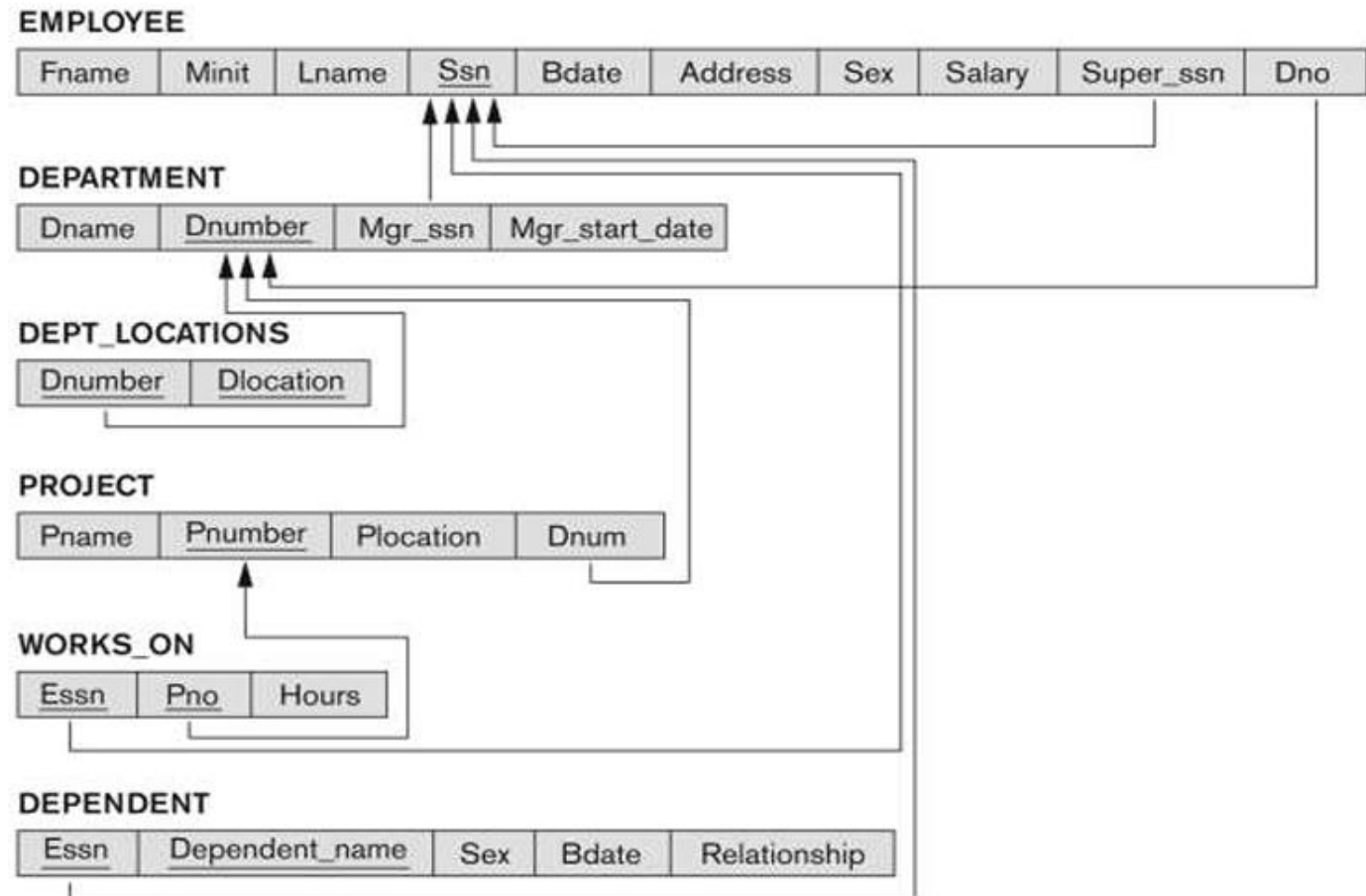
How does the **SELECT** statement eliminates duplicates ?
Automatically or not?

Database State for COMPANY

- All examples discussed below refer to the COMPANY database shown here.

Figure 5.7

Referential integrity constraints displayed on the COMPANY relational database schema.



Unary Relational Operations: PROJECT

- PROJECT Operation is denoted by π (pi)
- This operation keeps certain *columns* (attributes) from a relation and discards the other columns.
 - PROJECT creates a vertical partitioning
 - The list of specified columns (attributes) is kept in each tuple
 - The other attributes in each tuple are discarded
- Example: To list each employee's first and last name and salary, the following is used:

$\pi_{\text{LNAME, FNAME, SALARY}}(\text{EMPLOYEE})$

Example of applying SELECT OPERATION:

For example, to list each employee's first and last name and salary, we can use the PROJECT operation as follows:

π Lname, Fname, Salary(EMPLOYEE)

Lname	Fname	Salary
Smith	John	30000
Wong	Franklin	40000
Zelaya	Alicia	25000
Wallace	Jennifer	43000
Narayan	Ramesh	38000
English	Joyce	25000
Jabbar	Ahmad	25000
Borg	James	55000

Consider the following PROJECT operation:

Π Sex, Salary(EMPLOYEE)

Sex	Salary
M	30000
M	40000
F	25000
F	43000
M	38000
M	25000
M	55000

appears only once, duplicates are eliminated.

Unary Relational Operations: PROJECT (cont.)

- The general form of the *project* operation is:
$$\pi_{\langle \text{attribute list} \rangle} (R)$$
 - π (pi) is the symbol used to represent the *project* operation
 - $\langle \text{attribute list} \rangle$ is the desired list of attributes from relation R.
- The project operation *removes any duplicate tuples*
 - This is because the result of the *project* operation must be a *set of tuples*
 - Mathematical sets *do not allow* duplicate elements.

Unary Relational Operations: PROJECT (contd.)

■ PROJECT Operation Properties

- The number of tuples in the result of projection $\pi_{\langle \text{list} \rangle}(R)$ is always less or equal to the number of tuples in R
 - If the list of attributes includes a *key* of R , then the number of tuples in the result of PROJECT is *equal* to the number of tuples in R
- PROJECT is *not* commutative $\longrightarrow \pi_{A,B}(\pi_A(R)) \neq \pi_A(\pi_{A,B}(R))$
 - $\pi_{\langle \text{list1} \rangle}(\pi_{\langle \text{list2} \rangle}(R)) = \pi_{\langle \text{list1} \rangle}(R)$ as long as $\langle \text{list2} \rangle$ contains the attributes in $\langle \text{list1} \rangle$

Basic Operations: Select σ

- Selects rows that satisfy the selection condition.
 - No duplicates in result (Why?)
 - Schema of result is identical to schema of input relation
- Selection predicates can include: $<$, $>$, $=$, $\neq$, and, or, not
 - Examples:
 - $\sigma_{Sid \neq 55516}(S1)$
 - $\sigma_{Name = 'Smith'}(S1)$
- Syntax: $\sigma_{Conditional}(Relation)$

SID	Name	Login	DoB	GPA
55515	Smith	smith@ccs	Jan 10,1990	3.82
55516	Jones	jones@hist	Feb 11, 1992	2.98
55517	Ali	ali@math	Sep 22, 1989	3.11
55518	Smith	smith@math	Nov 30, 1991	3.32

$$\sigma_{Sid > 55516}(S1)$$

SID	Name	Login	DoB	GPA
55517	Ali	ali@math	Sep 22, 1989	3.11
55518	Smith	smith@math	Nov 30, 1991	3.32

The **select operation (σ)** eliminates duplicates because operations are based on **set theory**, and sets do not allow duplicate elements.

Employees

EmployeeID	Name	Age
1	Alice	30
2	Bob	25
3	Charlie	28
1	Alice	30

select operation (σ) to find all employees with the name "Alice":
 $\sigma_{\text{Name}='Alice'}(\text{Employees})$

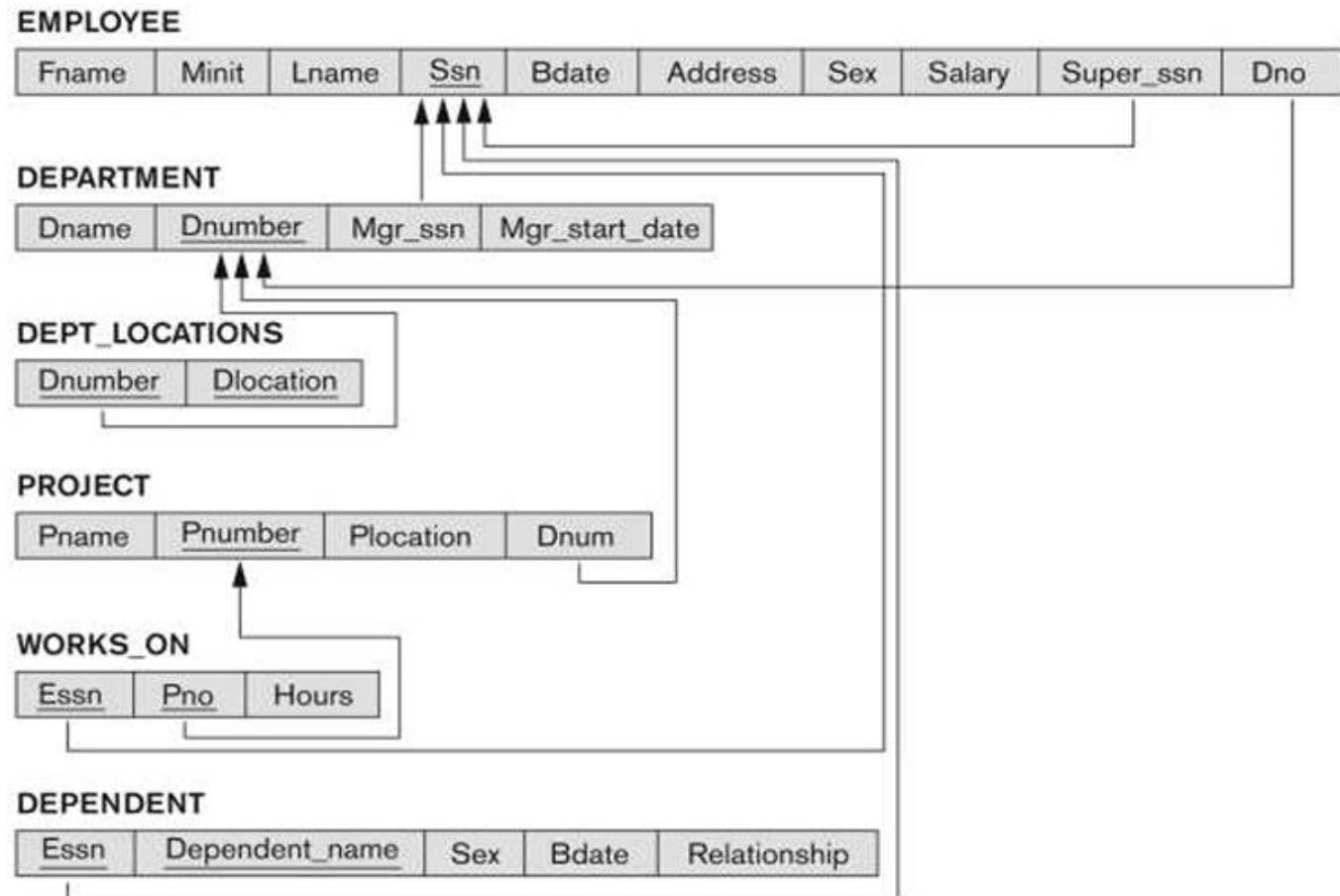
EmployeeID	Name	Age
1	Alice	30

Database State for COMPANY

- All examples discussed below refer to the COMPANY database shown here.

Figure 5.7

Referential integrity constraints displayed on the COMPANY relational database schema.



Unary Relational Operations: SELECT

- The SELECT operation (denoted by σ (sigma)) is used to select a *subset* of the tuples from a relation based on a **selection condition**.
 - The selection condition acts as a **filter**
 - Keeps only those tuples that satisfy the qualifying condition
 - Tuples satisfying the condition are *selected* whereas the other tuples are discarded (*filtered out*)
- Examples:
 - Select the EMPLOYEE tuples whose department number is 4:

$$\sigma_{DNO = 4} (EMPLOYEE)$$

- Select the employee tuples whose salary is greater than \$30,000:

$$\sigma_{SALARY > 30,000} (EMPLOYEE)$$

Example of applying SELECT OPERATION:

For example, To select the tuples for all employees who either work in department 4 and make over \$25,000 per year, **or** work in department 5 and make over \$30,000, we can specify the following SELECT operation:

$\sigma(\text{Dno}=4 \text{ AND Salary}>25000) \text{ OR } (\text{Dno}=5 \text{ AND Salary}>30000)(\text{EMPLOYEE})$

(a)

Fname	Minit	Lname	<u>Ssn</u>	Bdate	Address	Sex	Salary	Super_ssn	Dno
Franklin	T	Wong	333445555	1955-12-08	638 Voss, Houston, TX	M	40000	888665555	5
Jennifer	S	Wallace	987654321	1941-06-20	291 Berry, Bellaire, TX	F	43000	888665555	4
Ramesh	K	Narayan	666884444	1962-09-15	975 Fire Oak, Humble, TX	M	38000	333445555	5

In SQL, the SELECT condition is typically specified in the WHERE clause of a query.

For example, the following operation:

$\sigma_{Dno=4 \text{ AND } Salary > 25000} (EMPLOYEE)$

would correspond to the following SQL query:

```
SELECT *  
FROM EMPLOYEE  
WHERE Dno=4 AND Salary>25000;
```

Unary Relational Operations: SELECT

- In general, the *select* operation is denoted by $\sigma_{\langle \text{selection condition} \rangle}(R)$ where
 - the symbol σ (sigma) is used to denote the *select* operator
 - the selection condition is a Boolean (conditional) expression specified on the attributes of relation R
 - tuples that make the condition **true** are selected
 - appear in the result of the operation
 - tuples that make the condition **false** are filtered out
 - discarded from the result of the operation

Unary Relational Operations: SELECT (continued)

■ SELECT Operation Properties

- The SELECT operation $\sigma_{\langle \text{selection condition} \rangle}(R)$ produces a relation S that has the same schema (same attributes) as R
- SELECT σ is commutative:
 - $\sigma_{\langle \text{condition1} \rangle}(\sigma_{\langle \text{condition2} \rangle}(R)) = \sigma_{\langle \text{condition2} \rangle}(\sigma_{\langle \text{condition1} \rangle}(R))$
- Because of commutativity property, a cascade (sequence) of SELECT operations may be applied in any order:
 - $\sigma_{\langle \text{cond1} \rangle}(\sigma_{\langle \text{cond2} \rangle}(\sigma_{\langle \text{cond3} \rangle}(R))) = \sigma_{\langle \text{cond2} \rangle}(\sigma_{\langle \text{cond3} \rangle}(\sigma_{\langle \text{cond1} \rangle}(R)))$
- A cascade of SELECT operations may be replaced by a single selection with a conjunction of all the conditions:
 - $\sigma_{\langle \text{cond1} \rangle}(\sigma_{\langle \text{cond2} \rangle}(\sigma_{\langle \text{cond3} \rangle}(R))) = \sigma_{\langle \text{cond1} \rangle \text{ AND } \langle \text{cond2} \rangle \text{ AND } \langle \text{cond3} \rangle}(R)$
- The number of tuples in the result of a SELECT is less than (or equal to) the number of tuples in the input relation R

Operator composition example.

Select and Project

SID	Name	Login	DoB	GPA
55515	Smith	smith@ccs	Jan 10, 1990	3.82
55516	Jones	jones@hist	Feb 11, 1992	2.98
55517	Ali	ali@math	Sep 22, 1989	3.11
55518	Smith	smith@math	Nov 30, 1991	3.32

$\pi_{Sid, Name}(\sigma_{Sid > 55516}(S1))$

SID	Name
55517	Ali
55518	Smith

Schema of the Resulting Table

- Union, intersection, and difference operators
 - the schemas of the two operands must be the same, so use that schema for the result.
- Selection operator
 - schema of the result is the same as the schema of the operand.
- Projection operator
 - list of attributes determines the schema.

Relational Algebra Expressions

- We may want to apply several relational algebra operations one after the other
 - Either we can write the operations as a single **relational algebra expression** by nesting the operations, or
 - We can apply one operation at a time and create **intermediate result relations**.
- In the latter case, we must give names to the relations that hold the intermediate results.

Single expression versus sequence of relational operations (Example)

- To retrieve the first name, last name, and salary of all employees who work in department number 5, we must apply a select and a project operation
- We can write a *single relational algebra expression* as follows:

- $\pi_{\text{FNAME, LNAME, SALARY}}(\sigma_{\text{DNO}=5}(\text{EMPLOYEE}))$

- OR We can explicitly show the *sequence of operations*, giving a name to each intermediate relation:

- $\text{DEP5_EMPS} \leftarrow \sigma_{\text{DNO}=5}(\text{EMPLOYEE})$

- $\text{RESULT} \leftarrow \pi_{\text{FNAME, LNAME, SALARY}}(\text{DEP5_EMPS})$

Union U

- Takes two input relations, which must be union-compatible:
 - Same number of fields.
 - 'Corresponding' fields have the same type.

S1 U S2

SID	Name	Login	DoB	GPA
55515	Smith	smith@ccs	Jan 10,1990	3.82
55516	Jones	jones@hist	Feb 11, 1992	2.98
55517	Ali	ali@math	Sep 22, 1989	3.11
55518	Smith	smith@math	Nov 30, 1991	3.32
55515	Chen	chen@ccs	Jan 10,1990	3.01
55519	Alton	alton@hist	Jun 11, 1992	2.07

S1

SID	Name	Login	DoB	GPA
55515	Smith	smith@ccs	Jan 10,1990	3.82
55516	Jones	jones@hist	Feb 11, 1992	2.98
55517	Ali	ali@math	Sep 22, 1989	3.11
55518	Smith	smith@math	Nov 30, 1991	3.32

S2

SID	Name	Login	DoB	GPA
55515	Chen	chen@ccs	Jan 10,1990	3.01
55519	Alton	alton@hist	Jun 11, 1992	2.07
55517	Ali	ali@math	Sep 22, 1989	3.11
55518	Smith	smith@math	Nov 30, 1991	3.32

Relational Algebra Operations from Set Theory: UNION

■ UNION Operation

- Binary operation, denoted by $\cup$
- The result of $R \cup S$, is a relation that includes all tuples that are either in R or in S or in both R and S
- Duplicate tuples are eliminated
- The two operand relations R and S must be “type compatible” (or UNION compatible)
 - R and S must have same number of attributes
 - Each pair of corresponding attributes must be type compatible (have same or compatible domains)

Relational Algebra Operations from Set Theory: UNION

■ Example:

- To retrieve the social security numbers of all employees who either *work in department 5* (RESULT1 below) or *directly supervise an employee who works in department 5* (RESULT2 below)
- We can use the UNION operation as follows:

$\text{DEP5_EMPS} \leftarrow \sigma_{\text{DNO}=5} (\text{EMPLOYEE})$

$\text{RESULT1} \leftarrow \pi_{\text{SSN}}(\text{DEP5_EMPS})$

$\text{RESULT2}(\text{SSN}) \leftarrow \pi_{\text{SUPERSSN}}(\text{DEP5_EMPS})$

$\text{RESULT} \leftarrow \text{RESULT1} \cup \text{RESULT2}$

- The union operation produces the tuples that are in either RESULT1 or RESULT2 or both

Figure 8.3 Result of the UNION operation $\text{RESULT} \leftarrow \text{RESULT1} \cup \text{RESULT2}$.

RESULT1

Ssn
123456789
333445555
666884444
453453453

RESULT2

Ssn
333445555
888665555

RESULT

Ssn
123456789
333445555
666884444
453453453
888665555

Additional Relational Operations (continued)

■ OUTER UNION Operations

- The outer union operation was developed to take the union of tuples from two relations if the relations are *not type compatible*.
- This operation will take the union of tuples in two relations $R(X, Y)$ and $S(X, Z)$ that are **partially compatible**, meaning that only some of their attributes, say X , are type compatible.
- The attributes that are type compatible are represented only once in the result, and those attributes that are not type compatible from either relation are also kept in the result relation $T(X, Y, Z)$.

Additional Relational Operations (continued)

- Example: An outer union can be applied to two relations whose schemas are STUDENT(Name, SSN, Department, Advisor) and INSTRUCTOR(Name, SSN, Department, Rank).
 - Tuples from the two relations are matched based on having the same combination of values of the shared attributes— Name, SSN, Department.
 - If a student is also an instructor, both Advisor and Rank will have a value; otherwise, one of these two attributes will be null.
 - The result relation STUDENT_OR_INSTRUCTOR will have the following attributes:
- STUDENT_OR_INSTRUCTOR (Name, SSN, Department, Advisor, Rank)**

Intersection $\cap$

Occurs in S1 and S2

$$S1 \cap S2$$

SID	Name	Login	DoB	GPA
55517	Ali	ali@math	Sep 22, 1989	3.11
55518	Smith	smith@math	Nov 30, 1991	3.32

Set difference

Occurs in S1 but not in S2

$$S1 - S2$$

SID	Name	Login	DoB	GPA
55515	Smith	smith@ccs	Jan 10,1990	3.82
55516	Jones	jones@hist	Feb 11, 1992	2.98

S1

SID	Name	Login	DoB	GPA
55515	Smith	smith@ccs	Jan 10,1990	3.82
55516	Jones	jones@hist	Feb 11, 1992	2.98
55517	Ali	ali@math	Sep 22, 1989	3.11
55518	Smith	smith@math	Nov 30, 1991	3.32

S2

SID	Name	Login	DoB	GPA
55515	Chen	chen@ccs	Jan 10,1990	3.01
55519	Alton	alton@hist	Jun 11, 1992	2.07
55517	Ali	ali@math	Sep 22, 1989	3.11
55518	Smith	smith@math	Nov 30, 1991	3.32

Relational Algebra Operations from Set Theory

- Type Compatibility of operands is required for the binary set operation UNION $\cup$, (also for INTERSECTION $\cap$, and SET DIFFERENCE $-$, see next slides)
- $R1(A1, A2, \dots, An)$ and $R2(B1, B2, \dots, Bn)$ are type compatible if:
 - they have the same number of attributes, and
 - the domains of corresponding attributes are type compatible (i.e. $\text{dom}(Ai) = \text{dom}(Bi)$ for $i=1, 2, \dots, n$).
- The resulting relation for $R1 \cup R2$ (also for $R1 \cap R2$, or $R1 - R2$, see next slides) has the same attribute names as the *first* operand relation $R1$ (by convention)

Relational Algebra Operations from Set Theory: INTERSECTION

- INTERSECTION is denoted by $\cap$
- The result of the operation $R \cap S$, is a relation that includes all tuples that are in both R and S
 - The attribute names in the result will be the same as the attribute names in R
- The two operand relations R and S must be “type compatible”

Relational Algebra Operations from Set Theory: SET DIFFERENCE (cont.)

- SET DIFFERENCE (also called MINUS or EXCEPT) is denoted by –
- The result of $R - S$, is a relation that includes all tuples that are in R but not in S
 - The attribute names in the result will be the same as the attribute names in R
- The two operand relations R and S must be “type compatible”

Example to illustrate the result of UNION, INTERSECT, and DIFFERENCE

Figure 8.4 The set operations UNION, INTERSECTION, and MINUS. (a) Two union-compatible relations. (b) $\text{STUDENT} \cup \text{INSTRUCTOR}$. (c) $\text{STUDENT} \cap \text{INSTRUCTOR}$. (d) $\text{STUDENT} - \text{INSTRUCTOR}$. (e) $\text{INSTRUCTOR} - \text{STUDENT}$.

(a) STUDENT

Fn	Ln
Susan	Yao
Ramesh	Shah
Johnny	Kohler
Barbara	Jones
Amy	Ford
Jimmy	Wang
Ernest	Gilbert

INSTRUCTOR

Fname	Lname
John	Smith
Ricardo	Browne
Susan	Yao
Francis	Johnson
Ramesh	Shah

(b)

Fn	Ln
Susan	Yao
Ramesh	Shah
Johnny	Kohler
Barbara	Jones
Amy	Ford
Jimmy	Wang
Ernest	Gilbert
John	Smith
Ricardo	Browne
Francis	Johnson

(c)

Fn	Ln
Susan	Yao
Ramesh	Shah

(d)

Fn	Ln
Johnny	Kohler
Barbara	Jones
Amy	Ford
Jimmy	Wang
Ernest	Gilbert

(e)

Fname	Lname
John	Smith
Ricardo	Browne
Francis	Johnson

Some properties of UNION, INTERSECT, and DIFFERENCE

- Notice that both union and intersection are *commutative* operations; that is
 - $R \cup S = S \cup R$, and $R \cap S = S \cap R$
- Both union and intersection can be treated as n-ary operations applicable to any number of relations as both are *associative* operations; that is
 - $R \cup (S \cup T) = (R \cup S) \cup T$
 - $(R \cap S) \cap T = R \cap (S \cap T)$
- The minus operation is not commutative; that is, in general
 - $R - S \neq S - R$

Cross-Product x

Each row within S1 is paired with each row of C1

S1 x C1

SID	Name	Login	DoB	GPA	CID	Grade
55515	Smith	smith@ccs	Jan 10,1990	3.82	History 101	C
55515	Smith	smith@ccs	Jan 10,1990	3.82	Biology 220	A
55515	Smith	smith@ccs	Jan 10,1990	3.82	Anthro 320	B
55515	Smith	smith@ccs	Jan 10,1990	3.82	Music 101	A
55516	Jones	jones@hist	Feb 11, 1992	2.98	History 101	C
55516	Jones	jones@hist	Feb 11, 1992	2.98	Biology 220	A
55516	Jones	jones@hist	Feb 11, 1992	2.98	Anthro 320	B
55516	Jones	jones@hist	Feb 11, 1992	2.98	Music 101	A

S1

SID	Name	Login	DoB	GPA
55515	Smith	smith@ccs	Jan 10,1990	3.82
55516	Jones	jones@hist	Feb 11, 1992	2.98

C1

CId	Grade
History 101	C
Biology 220	A
Anthro 320	B
Music 101	A

Result schema has one field per field of S1 and C1, with field names 'inherited' if possible.

Relational Algebra Operations from Set Theory: CARTESIAN PRODUCT

- CARTESIAN (or CROSS) PRODUCT Operation
 - This operation is used to combine tuples from two relations in a combinatorial fashion.
 - Denoted by $R(A_1, A_2, \dots, A_n) \times S(B_1, B_2, \dots, B_m)$
 - Result is a relation Q with degree $n + m$ attributes:
 - $Q(A_1, A_2, \dots, A_n, B_1, B_2, \dots, B_m)$, in that order.
 - The resulting relation state has one tuple for each combination of tuples—one from R and one from S .
 - Hence, if R has n_R tuples (denoted as $|R| = n_R$), and S has n_S tuples, then $R \times S$ will have $n_R * n_S$ tuples.
 - The two operands do NOT have to be "type compatible"

Relational Algebra Operations from Set Theory: CARTESIAN PRODUCT (cont.)

- Generally, CROSS PRODUCT is not a meaningful operation
 - Can become meaningful when followed by other operations
- Example (not meaningful):
 - $FEMALE_EMPS \leftarrow \sigma_{SEX='F'}(EMPLOYEE)$
 - $EMP_NAMES \leftarrow \pi_{FNAME, LNAME, SSN}(FEMALE_EMPS)$
 - $EMP_DEPENDENTS \leftarrow EMP_NAMES \times DEPENDENT$
- EMP_DEPENDENTS will contain every combination of EMP_NAMES and DEPENDENT
 - whether or not they are actually related

Relational Algebra Operations from Set Theory: CARTESIAN PRODUCT (cont.)

- To keep only combinations where the DEPENDENT is related to the EMPLOYEE, we add a SELECT operation as follows
- Example (meaningful):
 - $FEMALE_EMPS \leftarrow \sigma_{SEX='F'}(EMPLOYEE)$
 - $EMP_NAMES \leftarrow \pi_{FNAME, LNAME, SSN}(FEMALE_EMPS)$
 - $EMP_DEPENDENTS \leftarrow EMP_NAMES \times DEPENDENT$
 - $ACTUAL_DEPS \leftarrow \sigma_{SSN=ESSN}(EMP_DEPENDENTS)$
 - $RESULT \leftarrow \pi_{FNAME, LNAME, DEPENDENT_NAME}(ACTUAL_DEPS)$
- RESULT will now contain the name of female employees and their dependents

Figure 8.5 The CARTESIAN PRODUCT (CROSS PRODUCT) operation.

FEMALE_EMPS

Fname	Minit	Lname	Ssn	Bdate	Address	Sex	Salary	Super_ssn	Dno
Alicia	J	Zelaya	999887777	1968-07-19	3321Castle, Spring, TX	F	25000	987654321	4
Jennifer	S	Wallace	987654321	1941-06-20	291Berry, Bellaire, TX	F	43000	888665555	4
Joyce	A	English	453453453	1972-07-31	5631 Rice, Houston, TX	F	25000	333445555	5

EMPNames

Fname	Lname	Ssn
Alicia	Zelaya	999887777
Jennifer	Wallace	987654321
Joyce	English	453453453

Figure 8.5 (continued) The CARTESIAN PRODUCT (CROSS PRODUCT) operation.

EMP_DEPENDENTS

Fname	Lname	Ssn	Essn	Dependent_name	Sex	Bdate	...
Alicia	Zelaya	999887777	333445555	Alice	F	1986-04-05	...
Alicia	Zelaya	999887777	333445555	Theodore	M	1983-10-25	...
Alicia	Zelaya	999887777	333445555	Joy	F	1958-05-03	...
Alicia	Zelaya	999887777	987654321	Abner	M	1942-02-28	...
Alicia	Zelaya	999887777	123456789	Michael	M	1988-01-04	...
Alicia	Zelaya	999887777	123456789	Alice	F	1988-12-30	...
Alicia	Zelaya	999887777	123456789	Elizabeth	F	1967-05-05	...
Jennifer	Wallace	987654321	333445555	Alice	F	1986-04-05	...
Jennifer	Wallace	987654321	333445555	Theodore	M	1983-10-25	...
Jennifer	Wallace	987654321	333445555	Joy	F	1958-05-03	...
Jennifer	Wallace	987654321	987654321	Abner	M	1942-02-28	...
Jennifer	Wallace	987654321	123456789	Michael	M	1988-01-04	...
Jennifer	Wallace	987654321	123456789	Alice	F	1988-12-30	...
Jennifer	Wallace	987654321	123456789	Elizabeth	F	1967-05-05	...
Joyce	English	453453453	333445555	Alice	F	1986-04-05	...
Joyce	English	453453453	333445555	Theodore	M	1983-10-25	...
Joyce	English	453453453	333445555	Joy	F	1958-05-03	...
Joyce	English	453453453	987654321	Abner	M	1942-02-28	...
Joyce	English	453453453	123456789	Michael	M	1988-01-04	...
Joyce	English	453453453	123456789	Alice	F	1988-12-30	...
Joyce	English	453453453	123456789	Elizabeth	F	1967-05-05	...

continued

Figure 8.5 (continued) The CARTESIAN PRODUCT (CROSS PRODUCT) operation.

ACTUAL_DEPENDENTS

Fname	Lname	Ssn	Essn	Dependent_name	Sex	Bdate	...
Jennifer	Wallace	987654321	987654321	Abner	M	1942-02-28	...

RESULT

Fname	Lname	Dependent_name
Jennifer	Wallace	Abner

Rename ρ

- Reassign the field names

$\rho(C(1 \rightarrow S1.sid, 6 \rightarrow C1.sid),$
 $S1 \times C1)$

C

S1.SID	Name	Login	DoB	GPA	C1.Sid	CID	Grade
55515	Smith	smith@ccs	Jan 10,1990	3.82	55515	History 101	C
55515	Smith	smith@ccs	Jan 10,1990	3.82	55515	Biology 220	A
55515	Smith	smith@ccs	Jan 10,1990	3.82	55515	Anthro 320	B
55515	Smith	smith@ccs	Jan 10,1990	3.82	55515	Music 101	A
55516	Jones	jones@hist	Feb 11, 1992	2.98	55516	History 101	C
55516	Jones	jones@hist	Feb 11, 1992	2.98	55516	Biology 220	A
55516	Jones	jones@hist	Feb 11, 1992	2.98	55516	Anthro 320	B
55516	Jones	jones@hist	Feb 11, 1992	2.98	55516	Music 101	A

S1

SID	Name	Login	DoB	GPA
55515	Smith	smith@ccs	Jan 10,1990	3.82
55516	Jones	jones@hist	Feb 11, 1992	2.98

C1

Sid	CId	Grade
55515	History 101	C
55516	Biology 220	A
55517	Anthro 320	B
55518	Music 101	A

Prepend the
name of the
original relation
to the fields
having a
collision

Naming
columns and
result set to C

Unary Relational Operations: RENAME

- The RENAME operator is denoted by ρ (rho)
- In some cases, we may want to *rename* the attributes of a relation or the relation name or both
 - Useful when a query requires multiple operations
 - Necessary in some cases (see JOIN operation later)

Unary Relational Operations: RENAME (continued)

- The general RENAME operation ρ can be expressed by any of the following forms:
 - $\rho_S(B_1, B_2, \dots, B_n)(R)$ changes both:
 - the relation name to S , *and*
 - the column (attribute) names to $B_1, B_1, \dots, B_n$
 - $\rho_S(R)$ changes:
 - the *relation name* only to S
 - $\rho_{(B_1, B_2, \dots, B_n)}(R)$ changes:
 - the *column (attribute) names* only to $B_1, B_1, \dots, B_n$

Unary Relational Operations: RENAME (continued)

- For convenience, we also use a *shorthand* for renaming attributes in an intermediate relation:
 - If we write:
 - $\text{RESULT} \leftarrow \pi_{\text{FNAME, LNAME, SALARY}}(\text{DEP5_EMPS})$
 - RESULT will have the *same attribute names* as DEP5_EMPS (same attributes as EMPLOYEE)
 - If we write:
 - $\text{RESULT}(\text{F, M, L, S, B, A, SX, SAL, SU, DNO}) \leftarrow \rho_{\text{RESULT}(\text{F.M.L.S.B,A,SX,SAL,SU,DNO})}(\text{DEP5_EMPS})$
 - The 10 attributes of DEP5_EMPS are *renamed* to F, M, L, S, B, A, SX, SAL, SU, DNO, respectively
- Note:** the $\leftarrow$ symbol is an assignment operator

Example of applying multiple operations and RENAME

(a)

Fname	Lname	Salary
John	Smith	30000
Franklin	Wong	40000
Ramesh	Narayan	38000
Joyce	English	25000

$\pi_{\text{Fname, Lname, Salary}}(\sigma_{\text{Dno}=5}(\text{EMPLOYEE}))$

↓

$\text{DEP5_EMPS} \leftarrow \sigma_{\text{Dno}=5}(\text{EMPLOYEE})$
 $\text{RESULT} \leftarrow \pi_{\text{Fname, Lname, Salary}}(\text{DEP5_EMPS})$

(b)

TEMP

Fname	Minit	Lname	Ssn	Bdate	Address	Sex	Salary	Super_ssn	Dno
John	B	Smith	123456789	1965-01-09	731 Fondren, Houston,TX	M	30000	333445555	5
Franklin	T	Wong	333445555	1955-12-08	638 Voss, Houston,TX	M	40000	888665555	5
Ramesh	K	Narayan	666884444	1962-09-15	975 Fire Oak, Humble,TX	M	38000	333445555	5
Joyce	A	English	453453453	1972-07-31	5631 Rice, Houston, TX	F	25000	333445555	5

R

First_name	Last_name	Salary
John	Smith	30000
Franklin	Wong	40000
Ramesh	Narayan	38000
Joyce	English	25000

$\text{TEMP} \leftarrow \sigma_{\text{Dno}=5}(\text{EMPLOYEE})$
 $\text{R}(\text{First_name, Last_name, Salary}) \leftarrow \pi_{\text{Fname, Lname, Salary}}(\text{TEMP})$

Figure 8.2

Results of a sequence of operations. (a) $\pi_{\text{Fname, Lname, Salary}}(\sigma_{\text{Dno}=5}(\text{EMPLOYEE}))$.
(b) Using intermediate relations and renaming of attributes.

Binary Relational Operations: JOIN

- JOIN Operation (denoted by $\bowtie$)
 - The sequence of CARTESIAN PRODUCT followed by SELECT is used quite commonly to identify and select related tuples from two relations
 - A special operation, called JOIN combines this sequence into a single operation
 - This operation is very important for any relational database with more than a single relation, because it allows us *combine related tuples* from various relations
 - The general form of a join operation on two relations $R(A_1, A_2, \dots, A_n)$ and $S(B_1, B_2, \dots, B_m)$ is:
$$R \bowtie_{\langle \text{join condition} \rangle} S$$
 - where R and S can be any relations that result from general *relational algebra expressions*.

Binary Relational Operations: JOIN (cont.)

- Example: Suppose that we want to retrieve the name of the manager of each department.
 - To get the manager's name, we need to combine each DEPARTMENT tuple with the EMPLOYEE tuple whose SSN value matches the MGRSSN value in the department tuple.
 - We do this by using the join  operation.
- $DEPT_MGR \leftarrow DEPARTMENT \bowtie_{MGRSSN=SSN} EMPLOYEE$
- MGRSSN=SSN is the join condition
 - Combines each department record with the employee who manages the department
 - The join condition can also be specified as DEPARTMENT.MGRSSN= EMPLOYEE.SSN

Figure 8.6 Result of the **JOIN** operation
 $\text{DEPT_MGR} \leftarrow \text{DEPARTMENT} \bowtie_{\text{Mgr_ssn=Ssn}} \text{EMPLOYEE}.$

DEPT_MGR

Dname	Dnumber	Mgr_ssn	...	Fname	Minit	Lname	Ssn	...
Research	5	333445555	...	Franklin	T	Wong	333445555	...
Administration	4	987654321	...	Jennifer	S	Wallace	987654321	...
Headquarters	1	888665555	...	James	E	Borg	888665555	...

Some properties of JOIN

- Consider the following JOIN operation:

$$\begin{array}{ccc} \blacksquare & R(A_1, A_2, \dots, A_n) & \bowtie & S(B_1, B_2, \dots, B_m) \\ & & R.A_i = S.B_j & \end{array}$$

- Result is a relation Q with degree $n + m$ attributes:
 - $Q(A_1, A_2, \dots, A_n, B_1, B_2, \dots, B_m)$, in that order.
- The resulting relation state has one tuple for each combination of tuples— r from R and s from S , but *only if they satisfy the join condition* $r[A_i] = s[B_j]$
- Hence, if R has n_R tuples, and S has n_S tuples, then the join result will generally have *less than* $n_R * n_S$ tuples.
- Only related tuples (based on the join condition) will appear in the result

Some properties of JOIN

- The general case of JOIN operation is called a Theta-join: $R \bowtie_{\theta} S$
- The join condition is called *theta*
- *Theta* can be any general boolean expression on the attributes of R and S; for example:
 - $R.A_i < S.B_j \text{ AND } (R.A_k = S.B_l \text{ OR } R.A_p < S.B_q)$
- Most join conditions involve one or more equality conditions “AND”ed together; for example:
 - $R.A_i = S.B_j \text{ AND } R.A_k = S.B_l \text{ AND } R.A_p = S.B_q$

Conditional Join

- Accepts a conditional
- Operation equivalent to:
- $S1 \bowtie_C C1 = \sigma_C(S1 \times C1)$
- Filters out tuples according to the conditional expression
- **$S1 \bowtie_{gpa > 3.0} C1$**

S1

SID	Name	Login	DoB	GPA
55515	Smith	smith@ccs	Jan 10,1990	3.82
55516	Jones	jones@hist	Feb 11, 1992	2.98

C1

CId	Grade
History 101	C
Biology 220	A
Anthro 320	B
Music 101	A

SID	Name	Login	DoB	GPA	CID	Grade
55515	Smith	smith@ccs	Jan 10,1990	3.82	History 101	C
55515	Smith	smith@ccs	Jan 10,1990	3.82	Biology 220	A
55515	Smith	smith@ccs	Jan 10,1990	3.82	Anthro 320	B
55515	Smith	smith@ccs	Jan 10,1990	3.82	Music 101	A

Conditional Join is equivalent to:
Cartesian project ($\times$)
Selection (σ)

Binary Relational Operations: EQUIJOIN

- EQUIJOIN Operation
- The most common use of join involves join conditions with *equality comparisons* only
- Such a join, where the only comparison operator used is =, is called an EQUIJOIN.
 - In the result of an EQUIJOIN we always have one or more pairs of attributes (whose names need not be identical) that have identical values in every tuple.
 - The JOIN seen in the previous example was an EQUIJOIN.

Equijoin $\bowtie_C$

- What does it do: performs a filtered Cartesian product
- Filters out tuples where the attribute that have the same name have a different value

- $S \bowtie_{S1.sid = C1.sid} C1$

S1

SID	Name	Login	DoB	GPA
55515	Smith	smith@ccs	Jan 10,1990	3.82
55516	Jones	jones@hist	Feb 11, 1992	2.98

C1

Sid	Cid	Grade
55515	History 101	C
55516	Biology 220	A
55517	Anthro 320	B
55518	Music 101	A

SID	Name	Login	DoB	GPA	CID	Grade
55515	Smith	smith@ccs	Jan 10,1990	3.82	History 101	C
55516	Jones	jones@hist	Feb 11, 1992	2.98	Biology 220	A

Only one copy
of Sid is in the
resultant
relation

Binary Relational Operations:

NATURAL JOIN Operation

- NATURAL JOIN Operation
 - Another variation of JOIN called NATURAL JOIN — denoted by $*$ — was created to get rid of the second (superfluous) attribute in an EQUIJOIN condition.
 - because one of each pair of attributes with identical values is superfluous
 - The standard definition of natural join requires that the two join attributes, or each pair of corresponding join attributes, *have the same name* in both relations
 - If this is not the case, a renaming operation is applied first.

Binary Relational Operations

NATURAL JOIN (continued)

- Example: To apply a natural join on the DNUMBER attributes of DEPARTMENT and DEPT_LOCATIONS, it is sufficient to write:
 - $DEPT_LOCS \leftarrow DEPARTMENT * DEPT_LOCATIONS$
- Only attribute with the same name is DNUMBER
- An implicit join condition is created based on this attribute:
 $DEPARTMENT.DNUMBER = DEPT_LOCATIONS.DNUMBER$
- Another example: $Q \leftarrow R(A,B,C,D) * S(C,D,E)$
 - The implicit join condition includes *each pair* of attributes with the same name, “AND”ed together:
 - $R.C = S.C \text{ AND } R.D = S.D$
 - Result keeps only one attribute of each such pair:
 - $Q(A,B,C,D,E)$

Example of NATURAL JOIN operation

Figure 8.7 Results of two natural join operations. (a) $\text{proj_dept} \leftarrow \text{project} \bowtie \text{dept}$. (b) $\text{dept_locs} \leftarrow \text{department} \bowtie \text{dept_locations}$.

(a)

PROJ_DEPT

Pname	Pnumber	Plocation	Dnum	Dname	Mgr_ssn	Mgr_start_date
ProductX	1	Bellaire	5	Research	333445555	1988-05-22
ProductY	2	Sugarland	5	Research	333445555	1988-05-22
ProductZ	3	Houston	5	Research	333445555	1988-05-22
Computerization	10	Stafford	4	Administration	987654321	1995-01-01
Reorganization	20	Houston	1	Headquarters	888665555	1981-06-19
Newbenefits	30	Stafford	4	Administration	987654321	1995-01-01

(b)

DEPT_LOCS

Dname	Dnumber	Mgr_ssn	Mgr_start_date	Location
Headquarters	1	888665555	1981-06-19	Houston
Administration	4	987654321	1995-01-01	Stafford
Research	5	333445555	1988-05-22	Bellaire
Research	5	333445555	1988-05-22	Sugarland
Research	5	333445555	1988-05-22	Houston

Natural Join

- What does it do: performs a filtered Cartesian product
- Filters out tuples where the attribute that have the same name have a different value

- **S1** ⋈ **C1**  No need to specify field list

S1

SID	Name	Login	DoB	GPA
55515	Smith	smith@ccs	Jan 10,1990	3.82
55516	Jones	jones@hist	Feb 11, 1992	2.98

C1

Sid	CId	Grade
55515	History 101	C
55516	Biology 220	A
55517	Anthro 320	B
55518	Music 101	A

SID	Name	Login	DoB	GPA	CID	Grade
55515	Smith	smith@ccs	Jan 10,1990	3.82	History 101	C
55516	Jones	jones@hist	Feb 11, 1992	2.98	Biology 220	A

Natural join is equivalent to:
 Cartesian project (x)
 Selection (σ)
 Projection (π)

Additional Relational Operations (continued)

- The OUTER JOIN Operation
 - In NATURAL JOIN and EQUIJOIN, tuples without a *matching* (or *related*) tuple are eliminated from the join result
 - Tuples with null in the join attributes are also eliminated
 - This amounts to loss of information.
 - A set of operations, called OUTER joins, can be used when we want to keep all the tuples in R, or all those in S, or all those in both relations in the result of the join, regardless of whether or not they have matching tuples in the other relation.

Additional Relational Operations (continued)

- The left outer **join** operation keeps every tuple in the first or left relation R in $R \bowtie\!\!\!\bowtie S$; if no matching tuple is found in S, then the attributes of S in the **join** result are filled or “padded” with null values.
- A similar operation, right outer **join**, keeps every tuple in the second or right relation S in the result of $R \bowtie\!\!\!\bowtie S$.
- A third operation, full outer **join**, denoted by $\bowtie\!\!\!\bowtie$ keeps all tuples in both the left and the right relations when no matching tuples are found, padding them with null values as needed.

Figure 8.12 The result of a LEFT OUTER JOIN operation.

RESULT

Fname	Minit	Lname	Dname
John	B	Smith	NULL
Franklin	T	Wong	Research
Alicia	J	Zelaya	NULL
Jennifer	S	Wallace	Administration
Ramesh	K	Narayan	NULL
Joyce	A	English	NULL
Ahmad	V	Jabbar	NULL
James	E	Borg	Headquarters

TEMP $\leftarrow$ (EMPLOYEE $\bowtie_{\text{Ssn=Mgr_ssn}}$ DEPARTMENT)
RESULT $\leftarrow \pi_{\text{Fname, Minit, Lname, Dname}}(\text{TEMP})$

Precedence of Relational Operators

- 1. $[\sigma, \pi, \rho]$ (highest).
- 2. $[X, \bowtie]$
- 3. $\cap$
- 4. $[\cup, -]$

Complete Set of Relational Operations

- The set of operations including SELECT σ , PROJECT π , UNION $\cup$, DIFFERENCE $-$, RENAME ρ , and CARTESIAN PRODUCT $\times$ is called a *complete set* because any other relational algebra expression can be expressed by a combination of these five operations.
- For example:
 - $R \cap S = (R \cup S) - ((R - S) \cup (S - R))$
 - $R \bowtie_{\langle \text{join condition} \rangle} S = \sigma_{\langle \text{join condition} \rangle} (R \times S)$

Binary Relational Operations: DIVISION

■ DIVISION Operation

- The division operation is applied to two relations
- $R(Z) \div S(X)$, where X subset Z . Let $Y = Z - X$ (and hence $Z = X \cup Y$); that is, let Y be the set of attributes of R that are not attributes of S .
- The result of DIVISION is a relation $T(Y)$ that includes a tuple t if tuples t_R appear in R with $t_R[Y] = t$, and with
 - $t_R[X] = t_s$ for every tuple t_s in S .
- For a tuple t to appear in the result T of the DIVISION, the values in t must appear in R in combination with *every* tuple in S .

Binary Relational Operations: DIVISION

In relational algebra, the **division** operator (denoted as $\div$) is used to find tuples in one relation (table) that are associated with all tuples in another relation.

e.g. We want to find the **SSNs of employees** who are assigned to **all the projects that Smith is assigned to**.

- a list of all the projects that Smith is assigned to.
- need to find employees who are assigned to **every one of those projects**.

i.e. By dividing the employees' project assignments by the projects Smith is assigned to, we can get those employees who meet the required condition.

Example of DIVISION

Figure 8.8
The DIVISION operation. (a) Dividing SSN_PNOS by SMITH_PNOS. (b) $T \leftarrow R \div S$.

*Find the SSNs of employees who are assigned to **all** the projects that Smith is assigned to.*

This can be expressed in relational algebra as:

$\pi_{SSN} (SSN_PNOS \div SMITH_PNOS)$ 

(a)

SSN_PNOS

Essn	Pno
123456789	1
123456789	2
666884444	3
453453453	1
453453453	2
333445555	2
333445555	3
333445555	10
333445555	20
999887777	30
999887777	10
987987987	10
987987987	30
987654321	30
987654321	20
888665555	20

SMITH_PNOS

Pno
1
2

SSNS

Ssn
123456789
453453453

(b)

R

A	B
a1	b1
a2	b1
a3	b1
a4	b1
a1	b2
a3	b2
a2	b3
a3	b3
a4	b3
a1	b4
a2	b4
a3	b4

S

A
a1
a2
a3

T

B
b1
b4

Examples of Division A/B

sno	pno
s1	p1
s1	p2
s1	p3
s1	p4
s2	p1
s2	p2
s3	p2
s4	p2
s4	p4

A

pno
p2

$B1$

sno
s1
s2
s3
s4

$A/B1$

pno
p2
p4

$B2$

sno
s1
s4

$A/B2$

pno
p1
p2
p4

$B3$

sno
s1

$A/B3$

Table 8.1 Operations of Relational Algebra

Table 8.1 Operations of Relational Algebra

OPERATION	PURPOSE	NOTATION
SELECT	Selects all tuples that satisfy the selection condition from a relation R .	$\sigma_{\langle \text{selection condition} \rangle}(R)$
PROJECT	Produces a new relation with only some of the attributes of R , and removes duplicate tuples.	$\pi_{\langle \text{attribute list} \rangle}(R)$
THETA JOIN	Produces all combinations of tuples from R_1 and R_2 that satisfy the join condition.	$R_1 \bowtie_{\langle \text{join condition} \rangle} R_2$
EQUIJOIN	Produces all the combinations of tuples from R_1 and R_2 that satisfy a join condition with only equality comparisons.	$R_1 \bowtie_{\langle \text{join condition} \rangle} R_2$, OR $R_1 \bowtie_{(\langle \text{join attributes 1} \rangle), (\langle \text{join attributes 2} \rangle)} R_2$
NATURAL JOIN	Same as EQUIJOIN except that the join attributes of R_2 are not included in the resulting relation; if the join attributes have the same names, they do not have to be specified at all.	$R_1 \star_{\langle \text{join condition} \rangle} R_2$, OR $R_1 \star_{(\langle \text{join attributes 1} \rangle), (\langle \text{join attributes 2} \rangle)} R_2$ OR $R_1 \star R_2$

Table 8.1 Operations of Relational Algebra (continued)

Table 8.1 Operations of Relational Algebra

OPERATION	PURPOSE	NOTATION
UNION	Produces a relation that includes all the tuples in R_1 or R_2 or both R_1 and R_2 ; R_1 and R_2 must be union compatible.	$R_1 \cup R_2$
INTERSECTION	Produces a relation that includes all the tuples in both R_1 and R_2 ; R_1 and R_2 must be union compatible.	$R_1 \cap R_2$
DIFFERENCE	Produces a relation that includes all the tuples in R_1 that are not in R_2 ; R_1 and R_2 must be union compatible.	$R_1 - R_2$
CARTESIAN PRODUCT	Produces a relation that has the attributes of R_1 and R_2 and includes as tuples all possible combinations of tuples from R_1 and R_2 .	$R_1 \times R_2$
DIVISION	Produces a relation $R(X)$ that includes all tuples $t[X]$ in $R_1(Z)$ that appear in R_1 in combination with every tuple from $R_2(Y)$, where $Z = X \cup Y$.	$R_1(Z) \div R_2(Y)$

Additional Relational Operations: Aggregate Functions and Grouping

- A type of request that cannot be expressed in the basic relational algebra is to specify mathematical **aggregate functions** on collections of values from the database.
- Examples of such functions include retrieving the average or total salary of all employees or the total number of employee tuples.
 - These functions are used in simple statistical queries that summarize information from the database tuples.
- Common functions applied to collections of numeric values include
 - SUM, AVERAGE, MAXIMUM, and MINIMUM.
- The COUNT function is used for counting tuples or values.

Aggregate Function Operation

- Use of the Aggregate Functional operation $\mathcal{F}$
 - $\mathcal{F}_{\text{MAX Salary}}$ (EMPLOYEE) retrieves the maximum salary value from the EMPLOYEE relation
 - $\mathcal{F}_{\text{MIN Salary}}$ (EMPLOYEE) retrieves the minimum Salary value from the EMPLOYEE relation
 - $\mathcal{F}_{\text{SUM Salary}}$ (EMPLOYEE) retrieves the sum of the Salary from the EMPLOYEE relation
 - $\mathcal{F}_{\text{COUNT SSN, AVERAGE Salary}}$ (EMPLOYEE) computes the count (number) of employees and their average salary
 - Note: count just counts the number of rows, without removing duplicates

Note: The symbol $\mathcal{F}$ is called the **script letter F** or **calligraphic F**. It's often used in mathematics, particularly in **set theory**, **functional analysis**, and **relational algebra**, to represent functions, operators, or specific mathematical concepts.

Using Grouping with Aggregation

Another common type of request involves grouping the tuples in a relation by the value of some of their attributes and then applying an aggregate function *independently to each group*. An example would be to group EMPLOYEE tuples by Dno, so that each group includes the tuples for employees working in the same department. We can then list each Dno value along with, say, the average salary of employees within the department, or the number of employees who work in the department.

We can define an AGGREGATE FUNCTION operation, using the symbol $\mathfrak{I}$ (pronounced *script F*)⁷, to specify these types of requests as follows:

$$\langle \text{grouping attributes} \rangle \mathfrak{I} \langle \text{function list} \rangle (R)$$

where $\langle \text{grouping attributes} \rangle$ is a list of attributes of the relation specified in R , and $\langle \text{function list} \rangle$ is a list of ($\langle \text{function} \rangle \langle \text{attribute} \rangle$) pairs. In each such pair, $\langle \text{function} \rangle$ is one of the allowed functions—such as SUM, AVERAGE, MAXIMUM, MINIMUM, COUNT—and $\langle \text{attribute} \rangle$ is an attribute of the relation specified by R . The resulting relation has the grouping attributes plus one attribute for each element in the function list. For example, to retrieve each department number, the number of employees in the department, and their average salary, while renaming the resulting attributes as indicated below, we write:

$$\rho_{R(\text{Dno, No_of_employees, Average_sal})} (\text{Dno } \mathfrak{I} \text{ COUNT Ssn, AVERAGE Salary (EMPLOYEE)})$$

Using Grouping with Aggregation

- The previous examples all summarized one or more attributes for a set of tuples
 - Maximum Salary or Count (number of) Ssn
- Grouping can be combined with Aggregate Functions
- Example: For each department, retrieve the DNO, COUNT SSN, and AVERAGE SALARY
- A variation of aggregate operation $\mathcal{F}$ allows this:
 - Grouping attribute placed to left of symbol
 - Aggregate functions to right of symbol
 - $\text{DNO } \mathcal{F}_{\text{COUNT SSN, AVERAGE Salary}} (\text{EMPLOYEE})$
- Above operation groups employees by DNO (department number) and computes the count of employees and average salary per department

Figure 8.10 The aggregate function operation.

- $\rho_{R(Dno, No_of_employees, Average_sal)}(Dno \Join COUNT Ssn, AVERAGE Salary (EMPLOYEE)).$
- $Dno \Join salary(EMPLOYEE).$
- $\Join COUNT Ssn, AVERAGE Salary(EMPLOYEE).$

R

(a)

Dno	No_of_employees	Average_sal
5	4	33250
4	3	31000
1	1	55000

(b)

Dno	Count_ssn	Average_salary
5	4	33250
4	3	31000
1	1	55000

(c)

Count_ssn	Average_salary
8	35125

Figure 7.1a Results of GROUP BY and HAVING (in SQL). Q24.

Fname	Minit	Lname	<u>Ssn</u>	...	Salary	Super_ssn	Dno
John	B	Smith	123456789	...	30000	333445555	5
Franklin	T	Wong	333445555		40000	888665555	5
Ramesh	K	Narayan	666884444		38000	333445555	5
Joyce	A	English	453453453		25000	333445555	5
Alicia	J	Zelaya	999887777		25000	987654321	4
Jennifer	S	Wallace	987654321		43000	888665555	4
Ahmad	V	Jabbar	987987987		25000	987654321	4
James	E	Bong	888665555		55000	NULL	1

Dno	Count (*)	Avg (Salary)
5	4	33250
4	3	31000
1	1	55000

Result of Q24

Grouping EMPLOYEE tuples by the value of Dno

Additional Relational Operations (continued)

- Recursive Closure Operations
 - Another type of operation that, in general, cannot be specified in the basic original relational algebra is **recursive closure**.
 - This operation is applied to a **recursive relationship**.
 - An example of a recursive operation is to retrieve all SUPERVISEES of an EMPLOYEE **e** at all levels — that is, all EMPLOYEE **e'** directly supervised by **e**; all employees **e''** directly supervised by each employee **e'**; all employees **e'''** directly supervised by each employee **e''**; and so on.

Additional Relational Operations (continued)

- Although it is possible to retrieve employees at each level and then take their union, we cannot, in general, specify a query such as “retrieve the supervisees of ‘James Borg’ at all levels” without utilizing a looping mechanism.
 - The SQL3 standard includes syntax for recursive closure.

Figure 8.11 A two-level recursive query.

SUPERVISION

(Borg's Ssn is 888665555)

(Ssn)

(Super_ssn)

Ssn1	Ssn2
123456789	333445555
333445555	888665555
999887777	987654321
987654321	888665555
666884444	333445555
453453453	333445555
987987987	987654321
888665555	null

RESULT1

Ssn
333445555
987654321

(Supervised by Borg)

RESULT2

Ssn
123456789
999887777
666884444
453453453
987987987

(Supervised by
Borg's subordinates)

RESULT

Ssn
123456789
999887777
666884444
453453453
987987987
333445555
987654321

(RESULT1 $\cup$ RESULT2)

Examples of Queries in Relational Algebra : Procedural Form

- **Q1: Retrieve the name and address of all employees who work for the 'Research' department.**

$\text{RESEARCH_DEPT} \leftarrow \sigma_{\text{DNAME}='Research'} (\text{DEPARTMENT})$

$\text{RESEARCH_EMPS} \leftarrow (\text{RESEARCH_DEPT} \bowtie_{\text{DNUMBER}=\text{DNOEMPLOYEE}} \text{EMPLOYEE})$

$\text{RESULT} \leftarrow \pi_{\text{FNAME}, \text{LNAME}, \text{ADDRESS}} (\text{RESEARCH_EMPS})$

- **Q6: Retrieve the names of employees who have no dependents.**

$\text{ALL_EMPS} \leftarrow \pi_{\text{SSN}} (\text{EMPLOYEE})$

$\text{EMPS_WITH_DEPS}(\text{SSN}) \leftarrow \pi_{\text{ESSN}} (\text{DEPENDENT})$

$\text{EMPS_WITHOUT_DEPS} \leftarrow (\text{ALL_EMPS} - \text{EMPS_WITH_DEPS})$

$\text{RESULT} \leftarrow \pi_{\text{LNAME}, \text{FNAME}} (\text{EMPS_WITHOUT_DEPS} * \text{EMPLOYEE})$

Examples of Queries in Relational Algebra – Single expressions

As a single expression, these queries become:

- **Q1: Retrieve the name and address of all employees who work for the 'Research' department.**

$$\pi_{\text{Fname, Lname, Address}} \left(\sigma_{\text{Dname} = \text{'Research'}} \left(\text{DEPARTMENT} \bowtie_{\text{Dnumber} = \text{Dno}} (\text{EMPLOYEE}) \right) \right)$$

- **Q6: Retrieve the names of employees who have no dependents.**

$$\pi_{\text{Lname, Fname}} \left(\left(\pi_{\text{Ssn}} (\text{EMPLOYEE}) - \rho_{\text{Ssn}} \left(\pi_{\text{Essn}} (\text{DEPENDENT}) \right) \right) * \text{EMPLOYEE} \right)$$

Examples of Queries in Relational Algebra

Query 2. For every project located in 'Stafford', list the project number, the controlling department number, and the department manager's last name, address, and birth date.

```
STAFFORD_PROJS  $\leftarrow \sigma_{Plocation='Stafford'}(PROJECT)$   
CONTR_DEPTS  $\leftarrow (STAFFORD\_PROJS \bowtie_{Dnum=Dnumber} DEPARTMENT)$   
PROJ_DEPT_MGRS  $\leftarrow (CONTR\_DEPTS \bowtie_{Mgr\_ssn=Ssn} EMPLOYEE)$   
RESULT  $\leftarrow \pi_{Pnumber, Dnum, Lname, Address, Bdate}(PROJ\_DEPT\_MGRS)$ 
```

In this example, we first select the projects located in Stafford, then join them with their controlling departments, and then join the result with the department managers. Finally, we apply a project operation on the desired attributes.

Query 3. Find the names of employees who work on *all* the projects controlled by department number 5.

```
DEPT5_PROJS  $\leftarrow \rho_{(Pno)}(\pi_{Pnumber}(\sigma_{Dnum=5}(PROJECT)))$   
EMP_PROJ  $\leftarrow \rho_{(Ssn, Pno)}(\pi_{Essn, Pno}(WORKS\_ON))$   
RESULT_EMP_SSNS  $\leftarrow EMP\_PROJ \div DEPT5\_PROJS$   
RESULT  $\leftarrow \pi_{Lname, Fname}(RESULT\_EMP\_SSNS * EMPLOYEE)$ 
```

In this query, we first create a table DEPT5_PROJS that contains the project numbers of all projects controlled by department 5. Then we create a table EMP_PROJ that holds (Ssn, Pno) tuples, and apply the division operation. Notice that we **renamed** the attributes so that they will be correctly used in the division operation. Finally, we join the result of the division, which holds only Ssn values, with the EMPLOYEE table to retrieve the Fname, Lname attributes from EMPLOYEE.

Examples of Queries in Relational Algebra

Query 4. Make a list of project numbers for projects that involve an employee whose last name is 'Smith', either as a worker or as a manager of the department that controls the project.

```
SMITHS(Essn)  $\leftarrow \pi_{\text{Ssn}} (\sigma_{\text{Lname}='Smith'}(\text{EMPLOYEE}))$   
SMITH_WORKER_PROJS  $\leftarrow \pi_{\text{Pno}}(\text{WORKS\_ON} * \text{SMITHS})$   
MGRS  $\leftarrow \pi_{\text{Lname, Dnumber}}(\text{EMPLOYEE} \bowtie_{\text{Ssn}=\text{Mgr\_ssn}} \text{DEPARTMENT})$   
SMITH_MANAGED_DEPTS(Dnum)  $\leftarrow \pi_{\text{Dnumber}} (\sigma_{\text{Lname}='Smith'}(\text{MGRS}))$   
SMITH_MGR_PROJS(Pno)  $\leftarrow \pi_{\text{Pnumber}}(\text{SMITH\_MANAGED\_DEPTS} * \text{PROJECT})$   
RESULT  $\leftarrow (\text{SMITH\_WORKER\_PROJS} \cup \text{SMITH\_MGR\_PROJS})$ 
```

In this query, we retrieved the project numbers for projects that involve an employee named Smith as a worker in SMITH_WORKER_PROJS. Then we retrieved the project numbers for projects that involve an employee named Smith as manager of the department that controls the project in SMITH_MGR_PROJS. Finally, we applied the

UNION operation on SMITH_WORKER_PROJS and SMITH_MGR_PROJS. As a single in-line expression, this query becomes:

```
 $\pi_{\text{Pno}} (\text{WORKS\_ON} \bowtie_{\text{Essn}=\text{Ssn}} (\pi_{\text{Ssn}} (\sigma_{\text{Lname}='Smith'}(\text{EMPLOYEE}))) \cup \pi_{\text{Pno}} ((\pi_{\text{Dnumber}} (\sigma_{\text{Lname}='Smith'}(\pi_{\text{Lname, Dnumber}}(\text{EMPLOYEE}))) \bowtie_{\text{Ssn}=\text{Mgr\_ssn}} \text{DEPARTMENT})) \bowtie_{\text{Dnum}=\text{ber}=\text{Dnum}} \text{PROJECT})$ 
```

Examples of Queries in Relational Algebra

Query 5. List the names of all employees with two or more dependents.

Strictly speaking, this query cannot be done in the *basic (original) relational algebra*. We have to use the AGGREGATE FUNCTION operation with the COUNT aggregate function. We assume that dependents of the *same* employee have *distinct* Dependent_name values.

$$\begin{aligned} T1(\text{Ssn}, \text{No_of_dependents}) &\leftarrow \text{Essn} \mathcal{S} \text{ COUNT Dependent_name}(\text{DEPENDENT}) \\ T2 &\leftarrow \sigma_{\text{No_of_dependents} > 2}(T1) \\ \text{RESULT} &\leftarrow \pi_{\text{Lname}, \text{Fname}}(T2 * \text{EMPLOYEE}) \end{aligned}$$

Query 6. Retrieve the names of employees who have no dependents.

This is an example of the type of query that uses the MINUS (SET DIFFERENCE) operation.

$$\begin{aligned} \text{ALL_EMPS} &\leftarrow \pi_{\text{Ssn}}(\text{EMPLOYEE}) \\ \text{EMPS_WITH_DEPS}(\text{Ssn}) &\leftarrow \pi_{\text{Essn}}(\text{DEPENDENT}) \\ \text{EMPS_WITHOUT_DEPS} &\leftarrow (\text{ALL_EMPS} - \text{EMPS_WITH_DEPS}) \\ \text{RESULT} &\leftarrow \pi_{\text{Lname}, \text{Fname}}(\text{EMPS_WITHOUT_DEPS} * \text{EMPLOYEE}) \end{aligned}$$

We first retrieve a relation with all employee Ssns in ALL_EMPS. Then we create a table with the Ssns of employees who have at least one dependent in EMPS_WITH_DEPS. Then we apply the SET DIFFERENCE operation to retrieve employees Ssns with no dependents in EMPS_WITHOUT_DEPS, and finally join this with EMPLOYEE to retrieve the desired attributes. As a single in-line expression, this query becomes:

$$\pi_{\text{Lname}, \text{Fname}}((\pi_{\text{Ssn}}(\text{EMPLOYEE}) - \rho_{\text{Ssn}}(\pi_{\text{Essn}}(\text{DEPENDENT}))) * \text{EMPLOYEE})$$

Examples of Queries in Relational Algebra

Query 7. List the names of managers who have at least one dependent.

$MGRS(Ssn) \leftarrow \pi_{Mgr_ssn}(DEPARTMENT)$

$EMPS_WITH_DEPS(Ssn) \leftarrow \pi_{Essn}(DEPENDENT)$

$MGRS_WITH_DEPS \leftarrow (MGRS \cap EMPS_WITH_DEPS)$

$RESULT \leftarrow \pi_{Lname, Fname}(MGRS_WITH_DEPS * EMPLOYEE)$

In this query, we retrieve the Ssns of managers in MGRS, and the Ssns of employees with at least one dependent in EMPS_WITH_DEPS, then we apply the SET INTERSECTION operation to get the Ssns of managers who have at least one dependent.

Relational Calculus

- Relational Calculus is an alternative way for expressing queries
 - Main feature: specify what you want, not how to get it
 - Many equivalent algebra “implementations” possible for a given calculus expression
- In short: SQL query without aggregation = relational calculus expression
- Relational algebra expression is similar to program, describing what operations to perform in what order

What is Relational Calculus?

- It is a formal language based upon predicate calculus
- It allows you to express the set of tuples you want from a database using a formula

Relational Calculus

- Comes in two flavors: Tuple relational calculus (TRC) and Domain relational calculus (DRC)
- TRC: Variables range over (i.e., get bound to) tuples.
- DRC: Variables range over domain elements (= attribute values)
- Both TRC and DRC are subsets of first-order logic
 - We use some short-hand notation to simplify formulas
- Expressions in the calculus are called *formulas*
- Answer tuple = assignment of constants to variables that make the formula evaluate to true

- Predicate Calculus, also known as First-Order Logic (FOL), is a formal system used in mathematics, logic, and computer science to express statements about **objects and their relationships**. It extends propositional logic by introducing **quantifiers** and **predicates**, allowing for more complex expressions.
- In mathematics, a **predicate** is a statement or assertion containing one or more variables, whose truth value depends on the values of those variables.
- e.g.: "x is greater than 5". This is a predicate because its truth depends on the value of 'x'. If $x = 6$, the statement is true, but if $x = 3$, it's false.
- In mathematical logic, **quantifiers** are words or symbols that specify the quantity or number of elements a statement applies to, either **"for all"** (universal quantifier) or **"there exists"** (existential quantifier).

- **Universal Quantifier ($\forall$):** Represents "for all" or "for every". It asserts that a property holds true **for every element** in a given set or domain.
- **Existential Quantifier ($\exists$):** Represents "there exists" or "there is at least one". It asserts that **at least one element** in a given set or domain satisfies a particular property.

Relational Calculus

- A **relational calculus** expression creates a new relation, which is specified in terms of variables that range over rows of the stored database relations (in **tuple calculus**) or over columns of the stored relations (in **domain calculus**).
- In a calculus expression, there is *no order of operations* to specify how to retrieve the query result—a calculus expression specifies only what information the result should contain.
 - This is the main distinguishing feature between relational algebra and relational calculus.

Relational Calculus (continued)

- Relational calculus is considered to be a **nonprocedural** or **declarative** language.
- This differs from relational algebra, where we must write a *sequence of operations* to specify a retrieval request; hence relational algebra can be considered as a **procedural** way of stating a query.

Tuple Relational Calculus

- The tuple relational calculus is based on specifying a number of tuple variables.
- Each tuple variable usually ranges over a particular database relation, meaning that the variable may take as its value any individual tuple from that relation.
- A simple tuple relational calculus query is of the form
$$\{t \mid \text{COND}(t)\}$$
 - where t is a tuple variable and $\text{COND}(t)$ is a conditional expression involving t .
 - The result of such a query is the set of all tuples t that satisfy $\text{COND}(t)$.

Tuple Relational Calculus (continued)

- Example: To find the first and last names of all employees whose salary is above \$50,000, we can write the following tuple calculus expression:

**{t.FNAME, t.LNAME | EMPLOYEE(t) AND
t.SALARY>50000}**

- The condition EMPLOYEE(t) specifies that the **range relation** of tuple variable t is EMPLOYEE.
- The first and last name (PROJECTION $\pi_{\text{FNAME, LNAME}}$) of each EMPLOYEE tuple t that satisfies the condition t.SALARY>50000 (SELECTION $\sigma_{\text{SALARY} > 50000}$) will be retrieved.

The Existential and Universal Quantifiers

- Two special symbols called quantifiers can appear in formulas; these are the universal quantifier ($\forall$) and the existential quantifier ($\exists$).
- Informally, a tuple variable t is bound if it is quantified, meaning that it appears in an $(\forall t)$ or $(\exists t)$ clause; otherwise, it is free.
- If F is a formula, then so are $(\exists t)(F)$ and $(\forall t)(F)$, where t is a tuple variable.
 - The formula $(\exists t)(F)$ is true if the formula F evaluates to true for some (at least one) tuple assigned to free occurrences of t in F ; otherwise $(\exists t)(F)$ is false.
 - The formula $(\forall t)(F)$ is true if the formula F evaluates to true for every tuple (in the universe) assigned to free occurrences of t in F ; otherwise $(\forall t)(F)$ is false.

The Existential and Universal Quantifiers (continued)

- $\forall$ is called the universal or “for all” quantifier because every tuple in “the universe of” tuples must make F true to make the quantified formula true.
- $\exists$ is called the existential or “there exists” quantifier because any tuple that exists in “the universe of” tuples may make F true to make the quantified formula true.

Example Query Using Existential Quantifier

- Retrieve the name and address of all employees who work for the 'Research' department. The query can be expressed as :
 $\{t.FNAME, t.LNAME, t.ADDRESS \mid EMPLOYEE(t) \text{ and } (\exists d) (DEPARTMENT(d) \text{ and } d.DNAME='Research' \text{ and } d.DNUMBER=t.DNO) \}$
- The only *free tuple variables* in a relational calculus expression should be those that appear to the left of the bar (|).
 - In above query, t is the only free variable; it is then *bound successively* to each tuple.
- If a tuple *satisfies the conditions* specified in the query, the attributes FNAME, LNAME, and ADDRESS are retrieved for each such tuple.
 - The conditions EMPLOYEE (t) and DEPARTMENT(d) specify the range relations for t and d.
 - The condition d.DNAME = 'Research' is a selection condition and corresponds to a SELECT operation in the relational algebra, whereas the condition d.DNUMBER = t.DNO is a JOIN condition.

Example Query Using Universal Quantifier

- Find the names of employees who work on *all* the projects controlled by department number 5. The query can be:
 $\{e.LNAME, e.FNAME \mid EMPLOYEE(e) \text{ and } ((\forall x)(\text{not}(\text{PROJECT}(x)) \text{ or } \text{not}(x.DNUM=5))$
 $\text{OR } ((\exists w)(\text{WORKS_ON}(w) \text{ and } w.ESSN=e.SSN \text{ and } x.PNUMBER=w.PNO))))\}$
- Exclude from the universal quantification all tuples that we are not interested in by making the condition true *for all such tuples*.
 - The first tuples to exclude (by making them evaluate automatically to true) are those that are not in the relation R of interest.
- In query above, using the expression **not(PROJECT(x))** inside the universally quantified formula evaluates to true all tuples x that are not in the PROJECT relation.
 - Then we exclude the tuples we are not interested in from R itself. The expression $\text{not}(x.DNUM=5)$ evaluates to true all tuples x that are in the project relation but are not controlled by department 5.
- Finally, we specify a condition that must hold on all the remaining tuples in R.
 $((\exists w)(\text{WORKS_ON}(w) \text{ and } w.ESSN=e.SSN \text{ and } x.PNUMBER=w.PNO)$

Languages Based on Tuple Relational Calculus

- The language **SQL** is based on tuple calculus. It uses the basic block structure to express the queries in tuple calculus:
 - **SELECT** <list of attributes>
 - **FROM** <list of relations>
 - **WHERE** <conditions>
- **SELECT** clause mentions the attributes being projected, the **FROM** clause mentions the relations needed in the query, and the **WHERE** clause mentions the selection as well as the join conditions.
 - SQL syntax is expanded further to accommodate other operations. (See Chapter 8).

Languages Based on Tuple Relational Calculus (continued)

- Another language which is based on tuple calculus is **QUEL** which actually uses the range variables as in tuple calculus. Its syntax includes:
 - **RANGE OF** <variable name> **IS** <relation name>
- Then it uses
 - **RETRIEVE** <list of attributes from range variables>
 - **WHERE** <conditions>
- This language was proposed in the relational DBMS **INGRES**. (system is currently still supported by Computer Associates – but the QUEL language is no longer there).

Languages Based on Tuple Relational Calculus (continued)

- Another language which is based on tuple calculus is **QUEL** which actually uses the range variables as in tuple calculus. Its syntax includes:
 - **RANGE OF** <variable name> **IS** <relation name>
- Then it uses
 - **RETRIEVE** <list of attributes from range variables>
 - **WHERE** <conditions>
- This language was proposed in the relational DBMS INGRES. (system is currently still supported by Computer Associates – but the QUEL language is no longer there).

Relational Calculus: Summary

- Relational calculus is non-operational
 - Users define queries in terms of what they want, not in terms of how to compute it. (Declarativeness.)
- Algebra and safe calculus have the same expressive power, leading to the notion of relational completeness.
- Relational calculus had big influence on the design of SQL and Query-by-Example

Practice Questions

Exercises (2)

- Which of the following relational algebra expressions are syntactically correct? What do they mean?

☐ STUDENTS.

☐ $\sigma_{\text{MAXPT} \neq 10}(\text{EXERCISES})$.

☐ $\pi_{\text{FIRST}}(\pi_{\text{LAST}}(\text{STUDENTS}))$.

☐ $\sigma_{\text{POINTS} \leq 5}(\sigma_{\text{POINTS} \geq 1}(\text{RESULTS}))$.

☐ $\sigma_{\text{POINTS}}(\pi_{\text{POINTS} = 10}(\text{RESULTS}))$.

Practice Questions

Example Database (1)

STUDENTS			
<u>SID</u>	FIRST	LAST	EMAIL
101	Ann	Smith	...
102	Michael	Jones	(null)
103	Richard	Turner	...
104	Maria	Brown	...

EXERCISES			
<u>CAT</u>	<u>ENO</u>	TOPIC	MAXPT
H	1	Rel. Algeb.	10
H	2	SQL	10
M	1	SQL	14

RESULTS			
<u>SID</u>	<u>CAT</u>	<u>ENO</u>	POINTS
101	H	1	10
101	H	2	8
101	M	1	12
102	H	1	9
102	H	2	9
102	M	1	10
103	H	1	5
103	M	1	7

Write the following queries in relational algebra:

- Print all homework results for Ann Smith (exercise number and points).
- Who has got the full points for a homework? Print first name, last name, and homework number.

This refers to the schema on Slide 6-4:

- STUDENTS(SID, FIRST, LAST, EMAIL^o)
- EXERCISES(CAT, ENO, TOPIC, MAXPT)
- RESULTS(SID→STUDENTS, (CAT, ENO)→EXERCISES, POINTS)

Practice Questions

Exercises

Write the following queries in relational algebra:

- Who got the most points in Homework 1?

Hint: Compute first students who did not get the most points, i.e. for which there is a student with more points. Then use set difference.

- Which students solved all exercises in the database?

This refers to the schema on Slide 6-4:

- STUDENTS(SID, FIRST, LAST, EMAIL^o)
- EXERCISES(CAT, ENO, TOPIC, MAXPT)
- RESULTS(SID→STUDENTS, (CAT, ENO)→EXERCISES, POINTS)

Practice Questions

Example 1. Consider the following SELECT-FROM-WHERE statement.

```
SELECT movieTitle
FROM StarsIn, MovieStar
WHERE starName = name AND birthdate = 1960
```

Its translation is as follows:

$$\pi_{\text{movieTitle}} \sigma_{\substack{\text{starName=name} \\ \wedge \text{birthdate=1960}}} (\text{StarsIn} \times \text{MovieStar}).$$

```
SELECT name FROM MovieStar
WHERE birthdate = 1960 AND name = starName
```

is translated into

$$\pi_{\text{movieTitle, movieYear, starName, name}} \sigma_{\substack{\text{birthdate=1960} \\ \wedge \text{name=starName}}} (\text{StarsIn} \times \text{MovieStar}).$$

Note how we have added the context relation **StarsIn** to the cartesian product, and how we have added the parameters **movieTitle**, **movieYear** and **starName** (the attributes of **StarsIn**) to the projection π . $\square$

Practice Questions

Example 8. Consider the following nested statement over the relations $R(A, B)$ and $S(C)$:

```
SELECT R1.A, R1.B
FROM R R1, S
WHERE EXISTS
  (SELECT R2.A, R2.B
   FROM R R_2
   WHERE R2.A = R1.B AND EXISTS
     (SELECT R3.A, R3.B
      FROM R R3
      WHERE R3.A = R2.B AND R3.B = S.C))
```

Let us denote the entire query by Q_1 ; the middle subquery by Q_2 ; and the inner subquery by Q_3 . Now assume that we are currently translating Q_2 . The subquery-free part of Q_2 is as follows:

```
SELECT *
FROM R R_2
WHERE R_2.A = R_1.B
```

Its translation is hence:

$$\sigma_{R_2.A=R_1.B}(\rho_{R_2}(R) \times \rho_{R_1}(R)) \quad (*)$$

Practice Questions

```
SELECT name, SUM(length)
FROM MovieExec, Movie
WHERE cert# = producerC#
GROUP BY name
HAVING MIN(year) < 1930
```

is thus translated into

$$\pi_{\text{name}, \text{SUM}(\text{length})} \sigma_{\text{MIN}(\text{year}) < 1930} \gamma_{\text{name}, \text{MIN}(\text{year}), \text{SUM}(\text{length})} \sigma_{\text{cert\#} = \text{producerC\#}} (\text{MovieExec} \times \text{Movie}).$$

□

Example 19. The subquery from Example 4:

```
SELECT SUM(B) FROM R
GROUP BY A
HAVING SUM(B) = C
```

is translated into

$$\pi_{C, \text{SUM}(B)} \sigma_{C = \text{SUM}(B)} \gamma_{C, A, \text{SUM}(B)} (R \times S).$$

Notice how we have added the context relation S to the cartesian product, and how we have added the parameter C to the operators γ and π . □

Practice Questions

Example 20. The following statement contains a subquery in its From-list:

```
SELECT movieTitle
FROM StarsIn, (SELECT name FROM MovieStar
                WHERE birthdate = 1960) M
WHERE starName = M.name
```

□

$\pi_{\text{movieTitle}} \sigma_{\text{starName}=M.\text{name}} (\text{StarsIn} \times \rho_M \pi_{\text{name}} \sigma_{\text{birthdate}=1960} (\text{MovieStar}))$.

□

Practice Questions : solve by yourself. Solutions given here may not be correct.

4a) Given the schema

EMP (Fname, Lname, SSN, Bdate, Address, Sex, Salary, SuperSSN, Dno)

DEPT T(Dname, Dnumber, MgrSSN, MGrstartdate)

DEPT-LOC (Dnumber,Dloc) PROJECT(Pname, Pnumber, Ploc,Dnum)

WORKS-ON (ESSN,PNo,Hours)

Give the relation algebra expression for the following:

1. List female employees from Dno=20 earning more than 50000

2. List 'CSE' department details.

Result $\leftarrow \sigma_{Dname='CSE'}(Department)$

3. Retrieve the first name, last name and salary of all employees who work in departmental number 50

Dep-Emps $\leftarrow \sigma_{Dno='50'}(Emp)$

4. Retrieve the name of the manager of each department.

Practice Questions

$\text{MGRS} \leftarrow \Pi_{\text{MgrSSn}} (\text{Department})$

$\text{Result} \leftarrow \Pi_{\text{Lname, Fname}} (\text{MGRS} \bowtie \text{EMP})$

5. Retrieve the name and address of all employees who work for sports department.

$\text{Sport-dept} \leftarrow \sigma_{\text{Dname}='sports'} (\text{Dept})$

$\text{Sport-Emps} \leftarrow (\text{Sport-dept} \bowtie_{\text{Dnumber=Dno}} \text{EMP})$

Practice Questions

4. (b) Consider the following relational schema and answer the (8)
following queries using relational algebra:

DEPARTMENT (Dnumber, Dname, MgrSSN, MgrStartDate)

PROJECT (Pnumber, Pname, Plocation, Dnumber)

EMPLOYEE (SSN, Name, Bdate, Addr, Sex, Salary, SuperSSN,
DNumber)

DEPENDENT (ESSN, Dependent_name, Sex, Bdate, Relationship)

- i) Retrieve the name and address of all employees who work for account department.
- ii) Retrieve the department number, number of employees and their average salary.
- iii) Retrieve the name and salary of the manager of each department.
- iv) List the names of managers who have atleast one dependent.

THANK YOU